Design & Analysis of Algorithms
Lecture 2

Basics of Algorithms and Mathematics

Computer Science & Informatics Department

SET THEORY

- A set is an unordered collection of distinct elements.
- A set is finite if it contains a finite number of elements; otherwise, the set is infinite.
- If X is a finite set, |x| , the cardinality of X, denotes the number of elements in X.
- If X is infinite set, we may write $|X| = \infty$
- The empty set, denoted $\emptyset$, is the unique set whose cardinality is 0.
- N = {0,1,2,3,….} Set of Natural Numbers.
- $x \in X$, $x \notin X$, | such that.
- Ex.{ n|n $\in$ N and n is odd}

SET NOTATION

$\{1, 4\}$	A set composed of the members 1 and 4
$\{x \mid x \text{ is a positive integer}\}$	A set definition using a **set former** Example: the set of all positive integers
$x \in \mathbf{P}$	x is a member of set **P**
$x \notin \mathbf{P}$	x is not a member of set **P**
$\emptyset$	The null or empty set
$\lvert\mathbf{P}\rvert$	Cardinality: size of set **P** or number of members for set **P**
$\mathbf{P} \subseteq \mathbf{Q}, \mathbf{Q} \supseteq \mathbf{P}$	Set **P** is included in set **Q**, set **P** is a subset of set **Q**, set **Q** is a superset of set **P**
$\mathbf{P} \cup \mathbf{Q}$	Set Union: all elements appearing in **P** OR **Q**
$\mathbf{P} \cap \mathbf{Q}$	Set Intersection: all elements appearing in **P** AND **Q**
$\mathbf{P} - \mathbf{Q}$	Set difference: all elements of set **P** NOT in set **Q**

- $X \subseteq Y$ " X is subset of Y "

  each elements of X belongs to Y.

- $X \subset Y$ " X is a proper subset of Y "

  means that $X \subseteq Y$ and moreover that there is at least one element in Y that does not belongs to X.

Representation of Sets

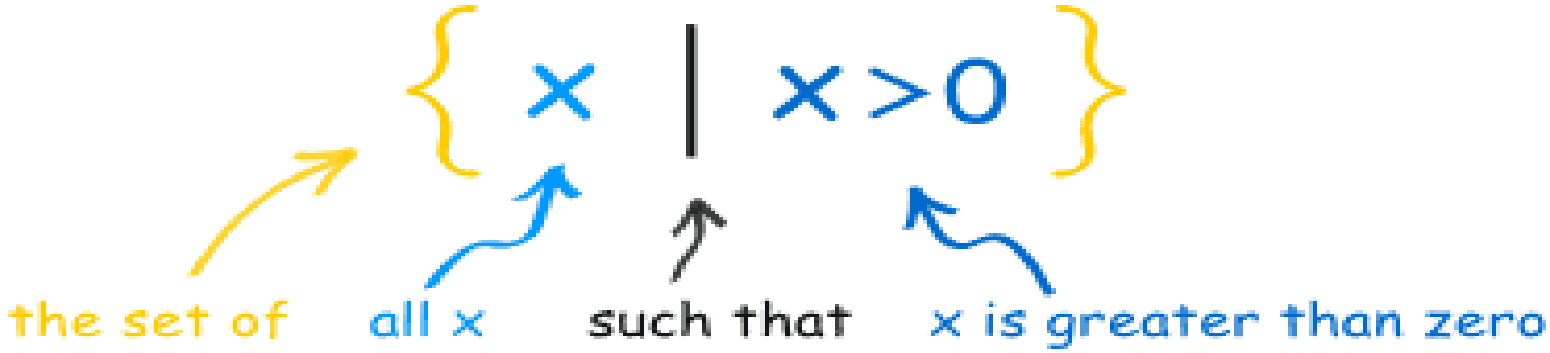
- **Three different ways to built a set**
  1) **Descriptive Form**
  2) **Set Builder Form (Comprehension Set)**
  3) **Tabular Form**

- **Descriptive Form**
- **E.g S= Set of all Prime Numbers (not useful from computer point of view)**

- **Tabular Form**
- **S={2,3,5,7,11,13……} ->:Little bit informative from Computer point of view but**
- **What about large sets (Finite, Infinite) cannot be written down.**

Representation of Sets

- Set Builder Notation (very useful)



It says *"the set of all x's, such that x is greater than 0".*

- Example:

$$primes == \{n : \mathbb{N} \mid n \geq 2 \land \forall m : 2..n-1 \bullet n \bmod m \neq 0\}$$

Partitioning of a Set

- A partition P of a Set A is a collection of { A_1, A_2, A_3, ….A_n } such that following rules are satisfied

Rules :

- ∀ Ai ∈ P, Ai ≠ Φ. (should not be empty)

  The family A does not contain the empty set

- Ai ∩ Aj = Φ ∀ i,j ∈ {1,2,3,….n} and i ≠ j. (do not overwrite)

  The intersection of any two distinct sets in P is empty

- A= A1 ∪ A2 ∪ A3 ∪….. An. (give same set)

  The union of the sets in P is equal to X

Applications areas of set partitioning:

1. Disk partitioning
2. Task partitioning on different machines

Sequences

It is sometimes necessary to record the order in which the objects are arranged.

- Data may be indexed by an ordered collection of keys.
- Messages may be stored in the order of arrival.
- Tasks may be performed in the order of importance.
- Names may be sorted in the order of alphabets and so on….

Definition:

- A group of elements in a specified order is called a **sequence**. A sequence can have repeated elements. i.e. multiset.
- Operations that apply on sequences include:
  - Merge
  - Filter
  - Extraction

Binary Relations

Definitions:

- **Relation** : Association of elements

If X and Y are two non empty sets then

$X \times Y = \{(x,y) \mid x \in X, y \in Y\}$ BINARY ASSOCIATION OR RELATION

CONSTRUCTION

If X={a,b} and Y={0,1} then

X xY = { (a,0), (a,1), (b,0),(b,1)}

Equivalence Relations

Definition

A relation, R, on X is **reflexive** if x R x for all x in X.

=== *being the same height as* is a reflexive relation: everything is the same height as itself

A relation, R, on X is **symmetric** if x R y implies that y R x.

====*being a cousin of* is a symmetric relation: if John is a cousin of Bill, then it is a logical consequence that Bill is a cousin of John .

A relation, R, on X is **transitive** if x R y and y R z imply that x R z.

==== if John is taller than Bill, and Bill is taller than Fred, then it is a logical consequence that John is taller than Fred.

If a relation is reflexive, symmetric and transitive then they are called equivalence relation ==== *identical* is an equivalence relation: if x is identical to y, and y is identical to z, then x is identical to z; if x is identical to y then y is identical to x; and x is identical to x.

Order Pair in form of Strings

Definition

A relation, R, over X,Y,Z is some subset X x Y x Z and so on.

For Example:

If ∑ ={0,1} then construct set of all strings of length 2 and 3

Solution:

Set of Length 2= ∑ x ∑ ={0,1}x{0,1}

={(0,0),(0,1),(1,0),(1,1)}

={00,01,10,11}

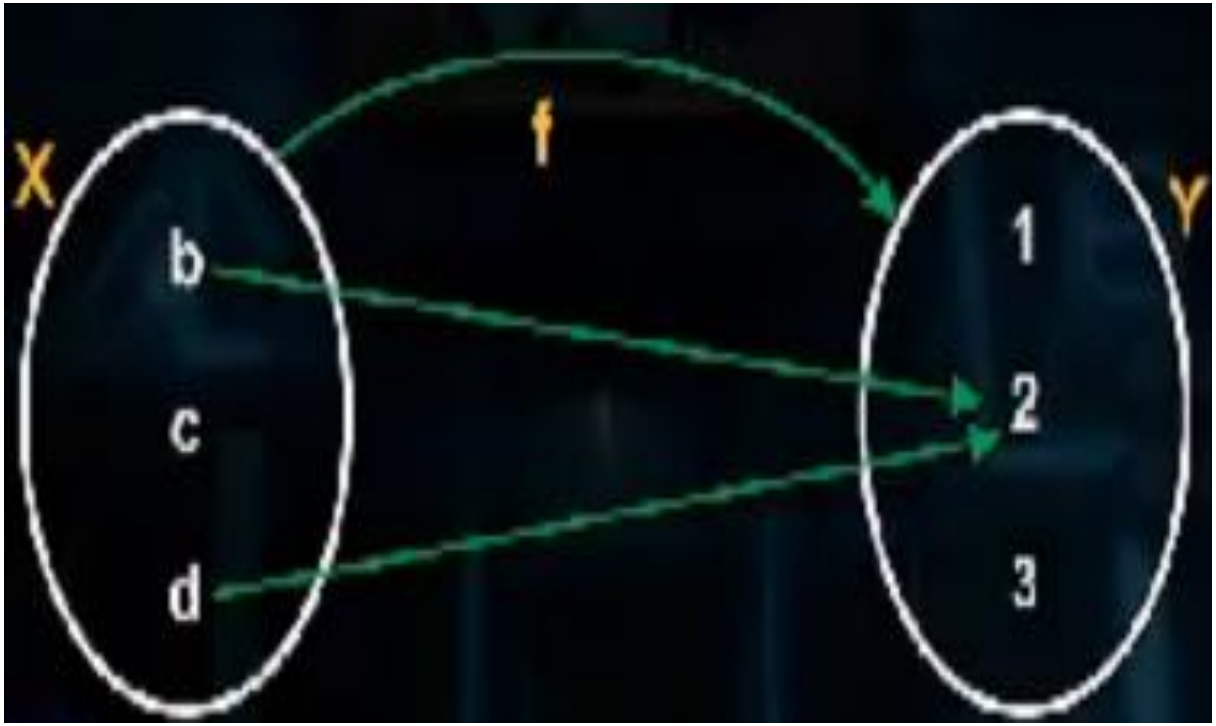
Set of Length 3= ∑ x ∑ x ∑ ={0,1}x{0,1} x{0,1}

={(0,0,0),(0,0,1),(0,1,0),(0,1,1),(1,0,0),(1,0,1),(1,1,0),(1,1,1)}

={000,001,010,011,100,101,110,111}

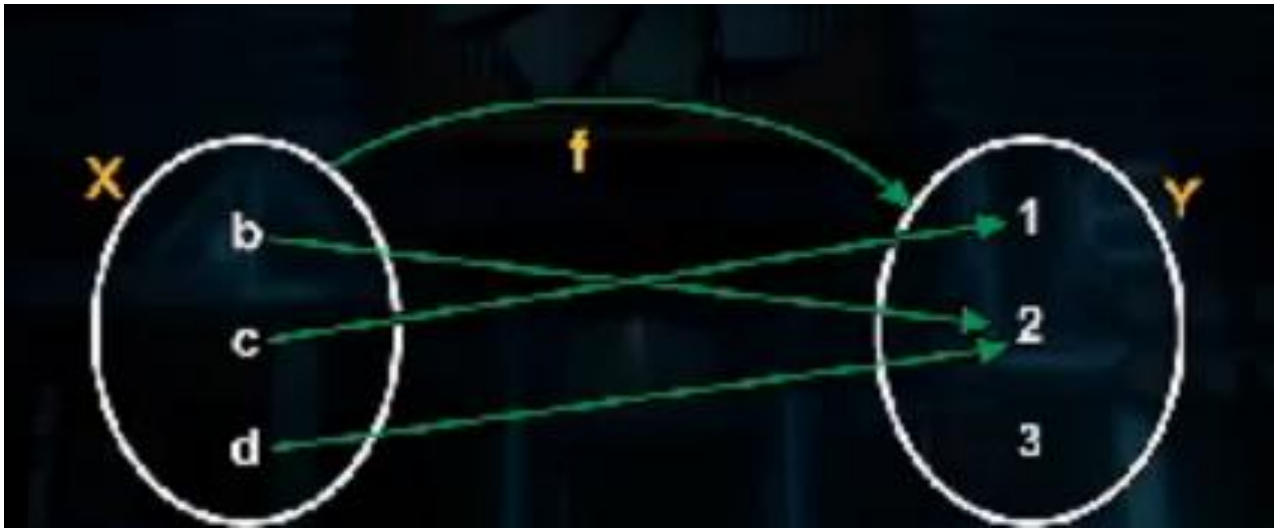
Partial Functions

A partial function is a relation that maps each element of X to at most one element of Y. X.Y denotes the set of all partial functions.



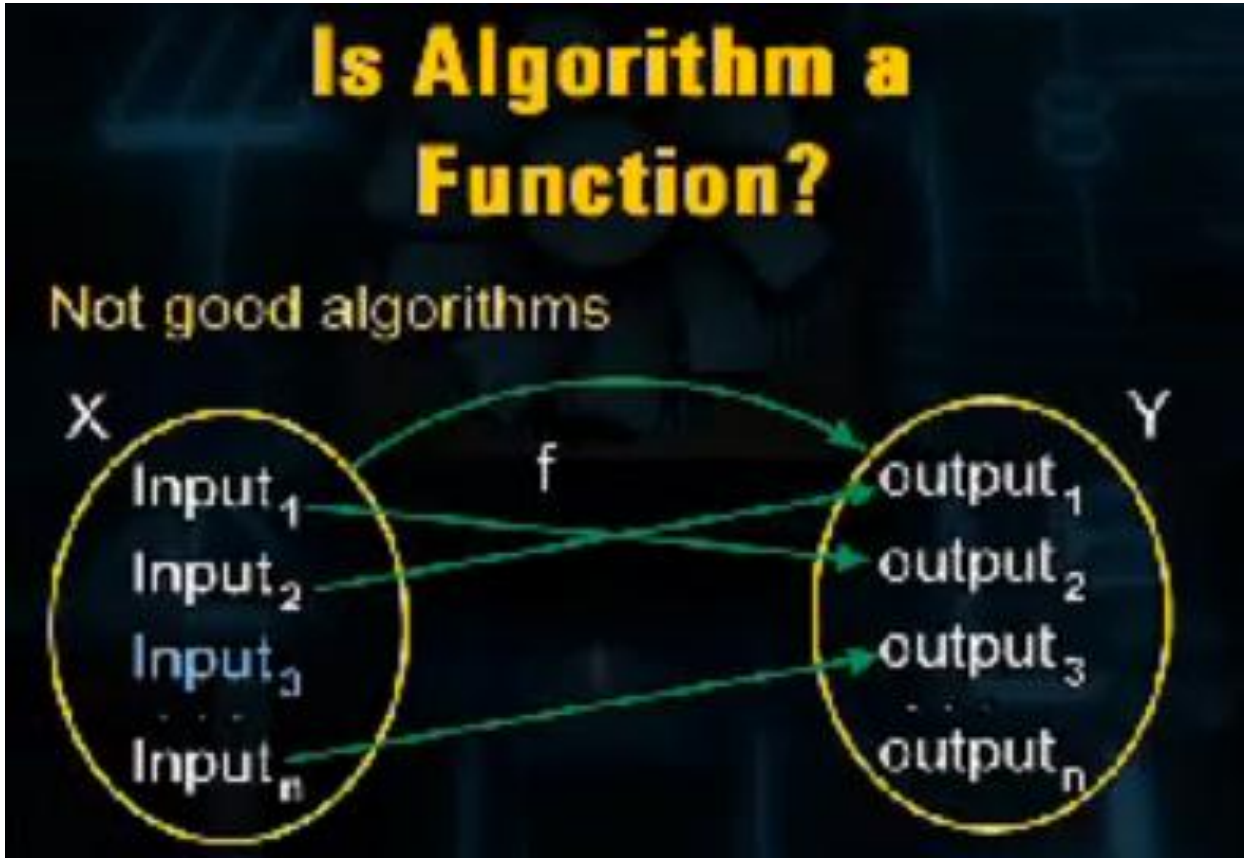
Functions

It is a relation in which each element of X is related to the element or image of Y Then partial function is a total function denoted by X ↗ Y . Here domain of function is X .Y



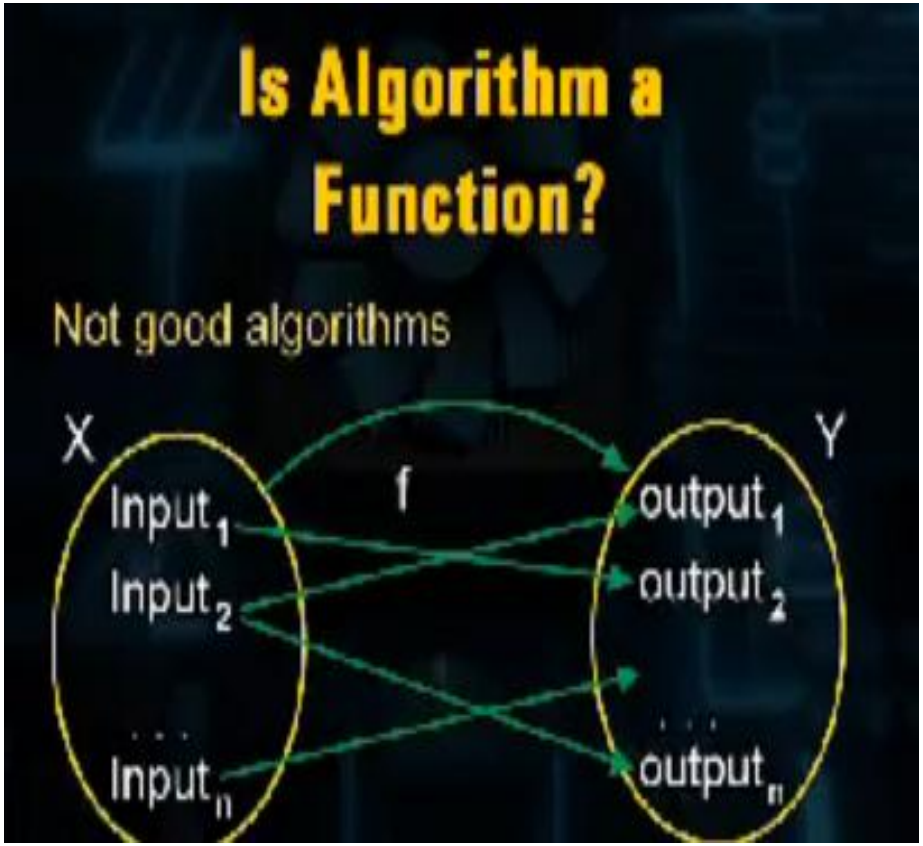
Relation VS Functions

Q) Does a single input Algorithm generate no output? Good or not



Relation VS Functions

Q) Does a single input Algorithm generate multiple outputs? Meaning generating different output in different timing? Good or not



Logarithms

A logarithm of base b for value y is the power to which b is raised to get y. Normally, this is written as $\log_b y = x$. Thus, if $\log_b y = x$ then $b^x = y$, and $b^{\log_b y} = y$.

Logarithms are used frequently by programmers. Here are typical uses.

Example: Many programs require an encoding for a collection of objects.

What is the minimum number of bits needed to represent n distinct code values?

The answer is $[\log_2 n]$ bits. For example, if you have 1000 codes to store, you will require at least $[\log_2 1000] = 10$ bits to have 1000 diffèrent codes (10 bits provide 1024 distinct code values).

Logarithms and Exponents

- Logarithms
  - properties of logarithms:
    - $\log_b(xy) = \log_b x + \log_b y$
    - $\log_b (x/y) = \log_b x - \log_b y$
    - $\log_b xa = a\log_b x$
    - $\log_b a = \log_x a/\log_x b$
- Exponents
  - properties of exponentials:
    - $a^{(b+c)} = a^b a^c$
    - $a^{bc} = (a^b)^c$
    - $a^b / a^c = a^{(b-c)}$
    - $b = a^{\log_a b}$
    - $b^c = a^{c*\log_a b}$
- Floor and ceiling functions
  - the largest integer less than or equal to x
  - the smallest integer greater than or equal to x

Important Summations

Arithmetic series

$$\sum_{i=1}^{n} i = 1 + 2 + \ldots + n$$

$$= \frac{n(n+1)}{2} = \Theta(n^2)$$

Quadratic series

$$\sum_{i=1}^{n} i^2 = 1 + 4 + 9 + \ldots + n^2$$

$$= \frac{2n^3 + 3n^2 + n}{6} = \Theta(n^3)$$

Geometric series

$$\sum_{i=1}^{n} x^i = 1 + x + x^2 + \ldots + x^n$$

$$= \frac{x^{(n+1)} - 1}{x - 1} = \Theta(n^2)$$

If $0 < x < 1$ then this is $\Theta(1)$, and if $x > 1$, then this is $\Theta(x^n)$.

Important Summations contd.

Harmonic series For $n \geq 0$

$$H_n = \sum_{i=1}^{n} \frac{1}{i}$$
$$= 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n} \approx \ln n$$
$$= \Theta(\ln n)$$

Proving Techniques:
1). Proof by Induction

- Many of the proofs when counting with integers can be done using **proof by induction**.
- Hence, it is important that we are comfortable in using induction for proofs.

- Proof by induction follows from the Axioms of Finite Induction that is used to show that a set of positive integers contains all the positive integers if the following hold.
  - ➢The set contains 1, and
  - ➢If the set contains a positive integer k, then it also contains k + 1.

1). Proof by Induction contd.

- There are THREE main things we need to be concerned with:
  - The induction hypothesis,
  - The base case
  - The induction step or the induction proof.

Example

- Prove the following by induction.

$$\sum_{i=1}^{n} i^3 = \left(\sum_{i=1}^{n} i\right)^2$$

Solution We need to identify the three basic elements of the proof.

Induction Hypothesis: It is what we need to prove, $\Sigma_{i=1}^{n} i^3 = (\Sigma_{i=1}^{n} i)^2$.

Base Case: We need to show that the induction hypothesis holds for the value of $n = 1$. This is easy to show. The left hand side of the equality is $\Sigma_{i=1}^{n} i^3$. When $n = 1$, the value of the left hand side is $\Sigma_{i=1}^{1} i^3 = 1^3 = 1$. The right hand side in this case is equal to $(\Sigma_{i=1}^{1} i)^2 = (1)^2 = 1$. Therefore, for the base case, the left hand side is equal in value to the right hand side. In other words, the induction hypothesis holds for the base case.

Induction Step: Here we need to show that if the induction hypothesis holds for an arbitrary positive integer value k (and for each positive integer smaller than k), it holds for the next higher integer $k + 1$. Let us assume that for some $k \geq 1$, the induction hypothesis holds. That is,

$$\Sigma_{i=1}^{k} i^3 = \left(\Sigma_{i=1}^{k} i\right)^2.$$

We need to show that

$$\Sigma_{i=1}^{k+1} i^3 = \left(\Sigma_{i=1}^{k+1} i\right)^2.$$

The proof follows. Let us start from the right hand side of the equation given immediately above.

$$
\begin{aligned}
\left(\sum_{i=1}^{k+1} i\right)^2 &= \left[\sum_{i=1}^{k} i + (k+1)\right]^2 \\
&= \left(\sum_{i=1}^{k} i\right)^2 + 2(k+1)\sum_{i=1}^{k} i + (k+1)^2 \\
&= \sum_{i=1}^{k} i^3 + 2(k+1)\frac{1}{2}k(k+1) + (k+1)^2 \\
&= \sum_{i=1}^{k} i^3 + (k+1)^2 k + (k+1)^2 \\
&= \sum_{i=1}^{k} i^3 + (k+1)(k+1) \\
&= \sum_{i=1}^{k} i^3 + (k+1)^3 \\
&= \sum_{i=1}^{k+1} i^3
\end{aligned}
\tag{1.1}
$$

Analysis and Complexity of Algorithms

Complexity of Algorithms

What is Complexity of Algorithms

- Storage Requirement
- Execution Time
- Implementation Details
- Hardware Requirement etc

- To measure all these resources is basically the complexity of Algorithms
- Complexity of an algorithm is analyzed in two perspectives: Time and Space.

Why ???

- Why we are measuring the complexity of Algorithms??.

- **Reason:** Not only to design algorithms but to design efficient type of algorithms which can finish in **finite** amount of time.

Analysis of Algorithms

- An *algorithm* is a finite set of precise instructions for performing a computation or for solving a problem.
- What is the goal of analysis of algorithms?
  - To compare algorithms mainly in terms of **running time** but also in terms of **other factors** (e.g., memory requirements, programmer's effort etc.)

- What do we mean by running time analysis?
  - **Determine how running time increases as the size of the problem increases**.

Input Size

- Input size (number of elements in the input)
  - size of an array
  - polynomial degree
  - # of elements in a matrix
  - # of bits in the binary representation of the input
  - vertices and edges in a graph

Efficiency Measurement

- Efficiency relative term
- Single algorithm : Polynomial Time Period (Efficient)

- Comparing Algorithms for finding their efficiencies

Types of Analysis

- Worst case (at most BIG O)
  - Provides an upper bound on running time
  - An absolute guarantee that the algorithm would not run longer, no matter what the inputs are
- Best case (at least Omega Ω)
  - Provides a lower bound on running time
  - Input is the one for which the algorithm runs the fastest

$$Lower\ Bound \leq Running\ Time \leq Upper\ Bound$$

- Average case (Theta Θ)
  - Provides a prediction about the running time
  - Assumes that the input is random

How do we compare algorithms?

- We need to define a number of objective measures.

(1) Compare execution times?

Not good: times are specific to a particular computer !!

(2) Count the number of statements executed?

Not good: number of statements vary with the programming language as well as the style of the individual programmer.

Ideal Solution

- Express running time as a growth function of the input size *n* (i.e., *f(n)*).
- Compare different functions corresponding to running times.
- Such an analysis is independent of machine time, programming style, etc.

Asymptotic Analysis

- To compare two algorithms with running times *f(n)* and *g(n)*, we need a **rough measure** that characterizes **how fast each function grows.**
- *Hint:* use *rate of growth*

- Compare functions in the limit, that is, **asymptotically!**

  (i.e., for large values of *n*)

Why asymptotic Notations

- Design efficient algorithm terminate in finite amount of Time/ acceptable amount of time.

Making use of Asymptotic notations for measuring running time of algorithm for large input size and measure its growth rate .

-

Asymptotic Notation

- Different types of asymptotic notations are used to represent the complexity of an algorithm.
  - **O** − Big Oh
  - **Ω** − Big omega
  - **θ** − Big theta
  - **o** − Little Oh
  - **ω** − Little omega

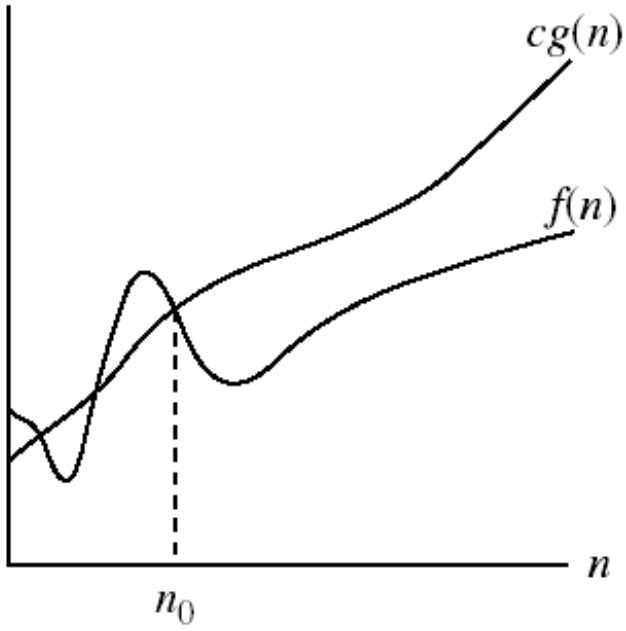
Asymptotic Notation

- O notation :Big-O is the formal method of expressing the upper bound of an algorithm's running time.
- It's a measure of the longest amount of time it could possibly take for the algorithm to complete.
- Formally, for non-negative functions, *f(n)* and *g(n)*, if there exists an integer n_0 and a constant $c > 0$ such that for all integers $n > n_0$, $f(n) \leq cg(n)$, then *f(n)* is Big O of *g(n).*

Asymptotic notations

- *O-notation*

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\}.$$



$g(n)$ is an ***asymptotic upper bound*** for $f(n)$.

Asymptotic Notation

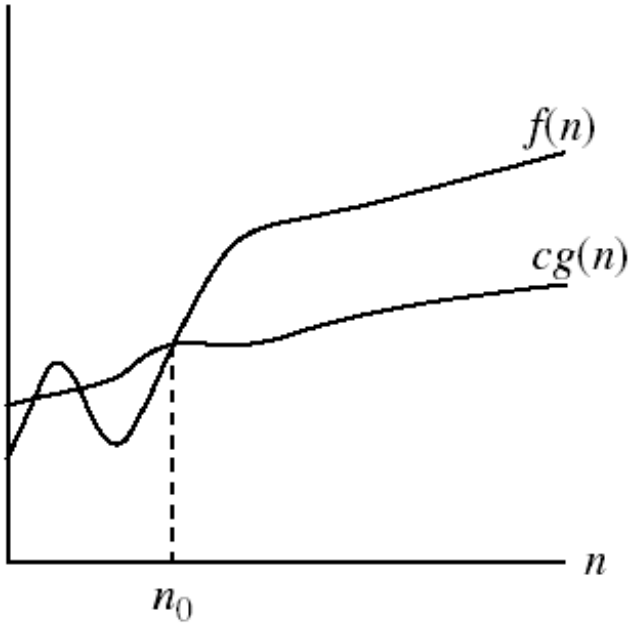
- ***Big-Omega Notation*** Ω
- This is almost the same definition as Big Oh, except that "$f(n) \geq cg(n)$"

- This makes $g(n)$ a lower bound function, instead of an upper bound function.
- It describes the **best that can happen** for a given data size.

- For non-negative functions, $f(n)$ and $g(n)$, if there exists an integer n_0 and a constant $c > 0$ such that for all integers $n > n_0$, $f(n) \geq cg(n)$, then $f(n)$ is omega of $g(n)$. This is denoted as "$f(n) = \Omega(g(n))$".

Asymptotic notations

$\Omega(g(n)) = \{f(n)$: there exist positive constants c and n_0 such that $0 \le cg(n) \le f(n)$ for all $n \ge n_0\}$.



Ω(g(n)) is the set of functions with larger or same order of growth as g(n)

$g(n)$ is an ***asymptotic lower bound*** for $f(n)$.

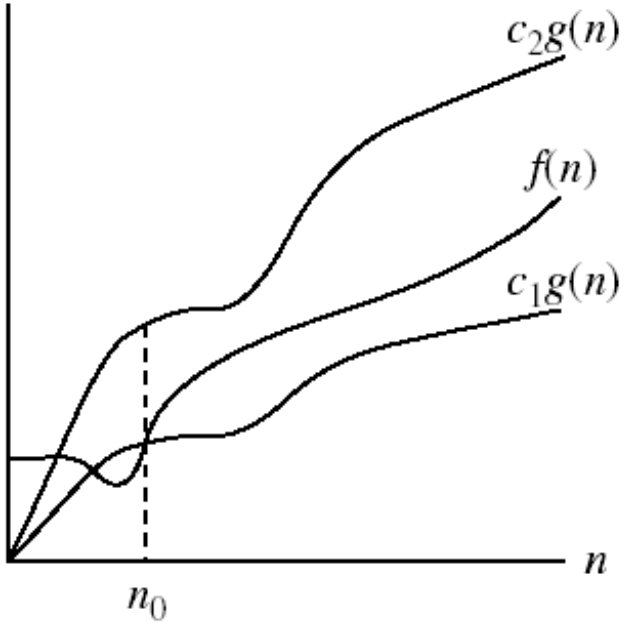
Asymptotic Notation

- **Theta Notation Θ**
- Theta Notation For non-negative functions, *f(n)* and *g(n)*, *f(n)* is theta of *g(n)* if and only if *f(n)* = *O(g(n))* and *f(n)* = *Ω(g(n))*. This is denoted as "*f(n)* = *Θ(g(n))*".

This is basically saying that the function, *f(n)* is bounded both from the top and bottom by the same function, *g(n)*.

Asymptotic notations

$\Theta(g(n)) = \{f(n)$: there exist positive constants c_1, c_2, and n_0 such that
$0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0\}$.



Θ(*g*(n)) is the set of functions with the same order of growth as *g*(n)

$g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

Asymptotic notations

- **Little-O Notation**
- For non-negative functions, *f(n)* and *g(n)*, *f(n)* is little o of *g(n)* if and only if *f(n) = O(g(n))*, but *f(n) ≠ Θ(g(n))*. This is denoted as "*f(n) = o(g(n))*".
- This represents a loose bounding version of Big O. *g(n)* bounds from the top, but it does not bound the bottom.
- **Little Omega Notation**
- For non-negative functions, *f(n)* and *g(n)*, *f(n)* is little omega of *g(n)* if and only if *f(n) = Ω(g(n))*, but *f(n) ≠ Θ(g(n))*. This is denoted as "*f(n) = ω(g(n))*".
- It bounds from the bottom, but not from the top.

Example

- Associate a "cost" with each statement.
- Find the "total cost“ by finding the total number of times each statement is executed.

Algorithm 1

	Cost
arr[0] = 0;	c_1
arr[1] = 0;	c_1
arr[2] = 0;	c_1
...	...
arr[N-1] = 0;	c_1

$c_1+c_1+...+c_1 = c_1 \times N$

Algorithm 2

	Cost
for(i=0; i<N; i++)	c_2
arr[i] = 0;	c_1

$(N+1) \times c_2 + N \times c_1 = (c_2 + c_1) \times N + c_2$

Another Example

- ***Algorithm 3*** *Cost*

```
sum = 0;                              c1
for(i=0; i<N; i++)                    c2
   for(j=0; j<N; j++)                 c2
      sum += arr[i][j];               c3
                                      ----------
```

$c_1 + c_2 \; x \; (N+1) + c_2 \; x \; N \; x \; (N+1) + c_3 \; x \; N^2$

Rate of Growth

- Consider the example of buying *elephants* and *goldfish:*

  Cost: cost_of_elephants + cost_of_goldfish

  Cost ~ cost_of_elephants (approximation)

- The low order terms in a function are relatively insignificant for **large** *n*

$$n^4 + 100n^2 + 10n + 50 \quad \sim \quad n^4$$

i.e., we say that $n^4 + 100n^2 + 10n + 50$ and n^4 have the same **rate of growth**

Big-O Notation

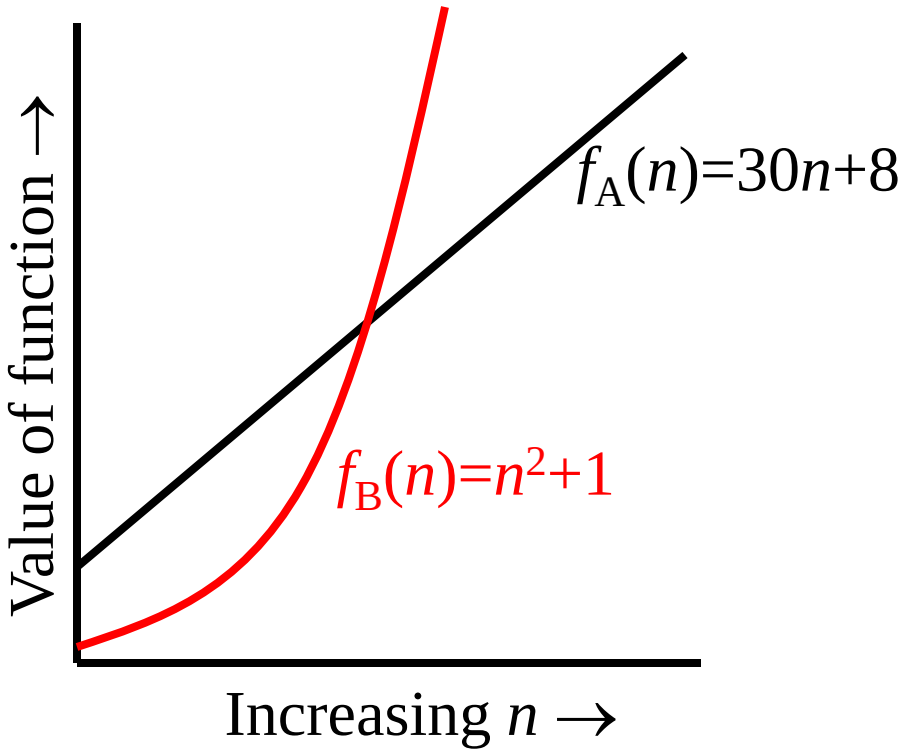
- We say $f_A(n)=30n+8$ is *order n*, or O (n)
  It is, at most, roughly *proportional* to *n*.

- $f_B(n)=n^2+1$ is *order* n^2, or $O(n^2)$. It is, at most, roughly proportional to n^2.

- In general, any $O(n^2)$ function is faster-growing than any $O(n)$ function.

Visualizing Orders of Growth

- On a graph, as you go to the right, a faster growing function eventually becomes larger...



More Examples …

- $n^4 + 100n^2 + 10n + 50$ is $O(n^4)$
- $10n^3 + 2n^2$ is $O(n^3)$
- $n^3 - n^2$ is $O(n^3)$
- constants
  - –10 is $O(1)$
  - –1273 is $O(1)$

Back to Our Example

Algorithm 1

	Cost
arr[0] = 0;	c_1
arr[1] = 0;	c_1
arr[2] = 0;	c_1
...	
arr[N-1] = 0;	c_1

$c_1+c_1+...+c_1 = c_1 \times N$

Algorithm 2

	Cost
for(i=0; i<N; i++)	c_2
arr[i] = 0;	c_1

$(N+1) \times c_2 + N \times c_1 = (c_2 + c_1) \times N + c_2$

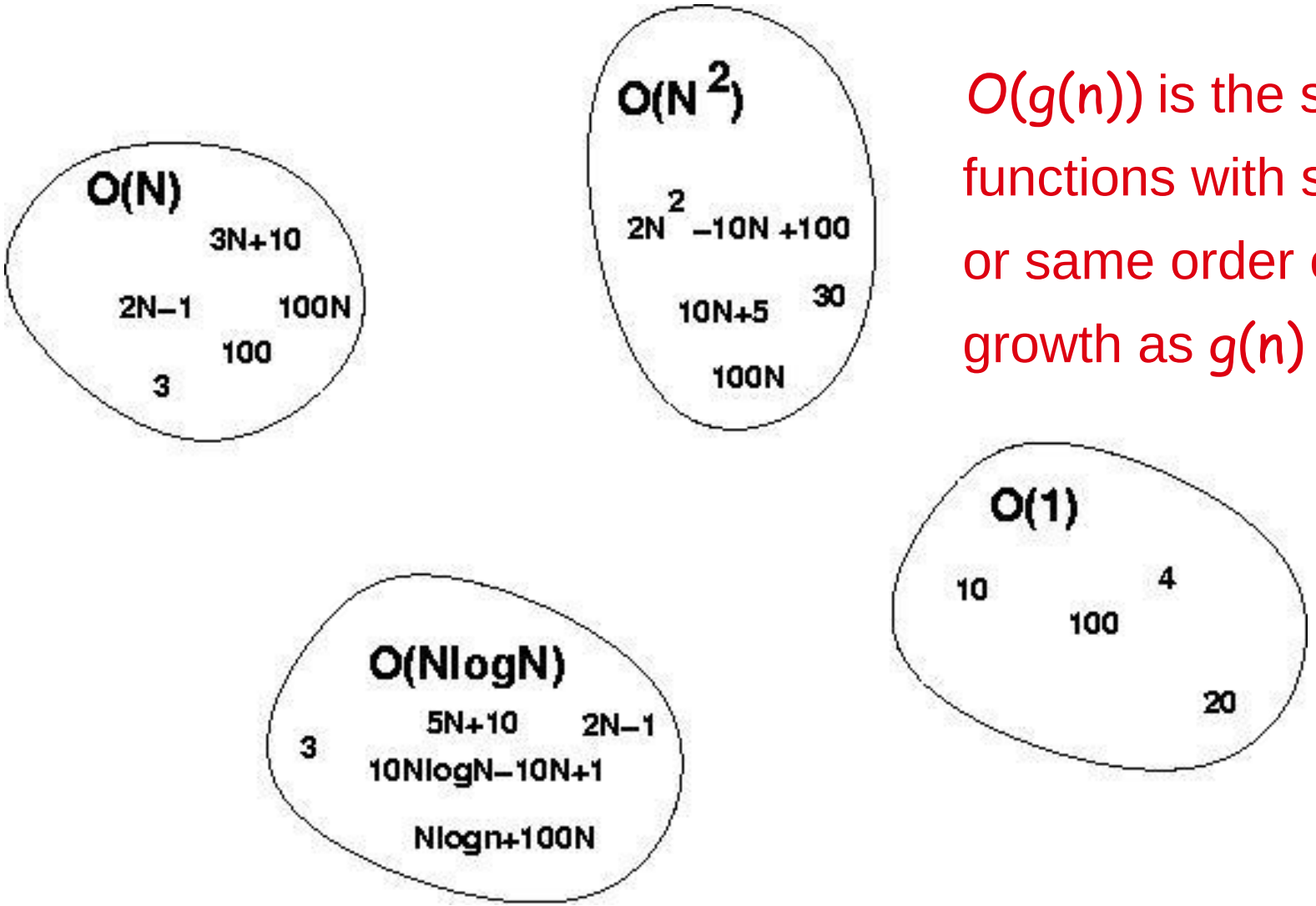
- Both algorithms are of the same order: *O(N)*

Example (cont'd)

Algorithm 3	***Cost***
sum = 0;	c_1
for(i=0; i<N; i++)	c_2
for(j=0; j<N; j++)	c_2
sum += arr[i][j];	c_3

$c_1 + c_2 \times (N+1) + c_2 \times N \times (N+1) + c_3 \times N^2 = O(N^2)$

Big-O Visualization



$O(g(n))$ is the set of functions with smaller or same order of growth as $g(n)$

Examples

- $2n^2 = O(n^3)$: $2n^2 \leq cn^3 \Rightarrow 2 \leq cn \Rightarrow c = 1$ and $n_0 = 2$

- $n^2 = O(n^2)$: $n^2 \leq cn^2 \Rightarrow c \geq 1 \Rightarrow c = 1$ and $n_0 = 1$

- $1000n^2+1000n = O(n^2)$:

$1000n^2+1000n \leq 1000n^2+ n^2 = 1001n^2 \Rightarrow c=1001$ and $n_0 = 1000$

- $n = O(n^2)$:

$n \leq cn^2 \Rightarrow cn \geq 1 \Rightarrow c = 1$ and $n_0 = 1$

More Examples

- Show that $30n+8$ is $O(n)$.
  - Show $\exists c, n_0$: $30n+8 \leq cn$, $\forall n > n_0$.
    - Let $c=31$, $n_0=8$. Assume $n>n_0=8$.

    Then
    - $cn = 31n = 30n + n > 30n+8$, so $30n+8 < cn$.

Big-O example, graphically

- Note 30*n*+8 isn't less than *n* *anywhere* (*n*>0).
- It isn't even less than 31*n* *everywhere*.
- But it *is* less than 31*n* <u>everywhere to the right of *n*=8</u>.

